Assignment 2 solution extra

1. Array navigation

- **Code**

```
module array();
    bit [11:0] my_array [4];
    initial begin
        my_array[0] = 12'h012;
        my_array[1] = 12'h345;
        my_array[2] = 12'h678;
        my_array[3] = 12'h9AB;
        //! traversing using for loop and print bits [5:4]
        $display("****traversing using for loop****");
        for(int i=0; i<$size(my_array); i++)
            $display("my_array[%0d][5:4]=%b", i, my_array[i][5:4]);
        //! traversing using foreach loop and print bits [5:4]
        $display("****traversing using foreach loop****");
        foreach (my_array[i]) begin
            $display("my_array[%0d][5:4]=%b", i, my_array[i][5:4]);
        end
        $stop;
    end
endmodule
```

- **Output result**

```
# ****traversing using for loop****
# my_array[0][5:4]=01
# my_array[1][5:4]=00
# my_array[2][5:4]=11
# my_array[3][5:4]=10
# ****traversing using foreach loop****
# my_array[0][5:4]=01
# my_array[1][5:4]=00
# my_array[2][5:4]=11
# my_array[3][5:4]=10
# ** Note: $stop    : array.sv(17)
#    Time: 0 ns  Iteration: 0  Instance: /array
# Break in Module array at array.sv line 17
# Stopped at array.sv line 17
# End time: 05:16:11 on Mar 22,2026, Elapsed time: 0:00:17
# Errors: 0, Warnings: 0
QuestaSim>
```

2. ALU

- **Design**

  Design behaves correctly and no bugs detected.

- **Verification plan**

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
ALU_1	When the reset is asserted, the output should be low	Direct at the start of the sim, then randomize with constraint that drive the reset to be off most of the simulation time	-	a checker in the testbench to make sure the output is correct
ALU_2	Try to randomize all the inputs for a while to and check the output against a golden model	Randomization under constrains on reset to be OFF most of the time	-	output checked against golden model

- **Package**

```
package ALU_PACKAGE;
    typedef enum logic [1:0] { Add, Sub, Not_A, ReductionOR_B } opcode_e;
    class ALU_R;
        rand bit reset;
        rand opcode_e opcode;
        rand bit [3:0] A,B;
        constraint c_dist {reset  dist {0:=98, 1:=2};}
    endclass
endpackage
```

- **Testbench**

```
import ALU_PACKAGE::*;
module ALU_tb ();
  //? local parameters
  localparam CLK_CYCLE = 2;
  //? class object handler creation
  ALU_R alu_obj;
  //? Interface signals
  logic reset, clk;
  logic signed [3:0] A, B;
  logic signed [4:0] C;
  opcode_e opcode;
  //? Variables
  int error_count, correct_count;
  //? DUT instantiation
  ALU_4_bit DUT_ALU (
      .clk(clk),
      .reset(reset),
      .Opcode(opcode),
      .A(A),
      .B(B),
      .C(C)
  );
  //? CLK generation
  always #(CLK_CYCLE / 2) clk = ~clk;
  //? Driver block
  initial begin
    //* initialization
    clk = 0;
    alu_obj = new();
    #(CLK_CYCLE) assert_reset();
    //! ALU Randomization test
    for (int i=0; i<100; i++) begin
      alu_obj.randomize();
      A = alu_obj.A;
      B = alu_obj.B;
      reset = alu_obj.reset;
      opcode = alu_obj.opcode;
      golden_model();
    end
    $display("error_counts = %0d, correct_counts = %0d", error_count, correct_count);
    $stop;
  end
  //? TASKS
  task assert_reset();
    reset = 1;
    check_result(0);
    reset = 0;
  endtask

  task check_result(input logic signed [4:0] expected_C);
    #CLK_CYCLE;
    if (expected_C == C) correct_count++;
    else begin
      error_count++;
      $display("xxxx ERROR at time %0t: the actual output result is not matching the expected result xxxx", $time);
      $display("A=%0b ,B=%0b, C_exp=%0b, C_actual=%0b, opcode=%0b, reset=%0b", A, B, expected_C, C, opcode, reset);
    end
  endtask
```

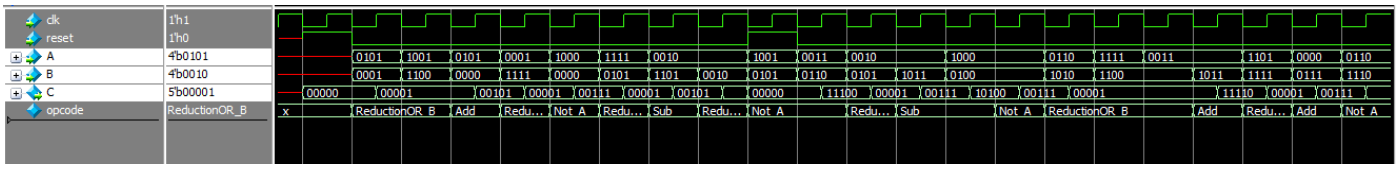
```
  task golden_model();
    logic signed [4:0] c_exp;
    if (reset) c_exp = 0;
    else begin
      case (opcode)
        Add: c_exp = A + B;
        Sub: c_exp = A - B;
        Not_A: c_exp = ~A;
        ReductionOR_B: c_exp = |B;
        default: c_exp = 0;
      endcase
    end
    check_result(c_exp);
  endtask
endmodule
```

- **DO file**

```
vlib work
vlog ALU.v ALU_PACKAGE.sv ALU_tb.sv +cover -covercells
vsim -voptargs=+acc work.ALU_tb -cover
coverage save ALU.ucdb -onexit -du ALU_4_bit
#do wave.do
log -r /*
run -all
coverage exclude -src ALU.v -line 26 -code s
coverage exclude -src ALU.v -line 26 -code b
quit -sim
vcover report ALU.ucdb -details -annotate -all -output cover_rpt.txt
```

- **Simulation results**

❖ **ALU_1 & ALU_2 tests**



- **Code coverage**

Default branch in the case statement has been excluded

❖ **Branch coverage**

```
Coverage Report by instance with details

================================================================================
=== Instance: /\ALU_tb#DUT_ALU
=== Design Unit: work.ALU_4_bit
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         6         6         0   100.00%

================================Branch Details================================
```

❖ **Statement coverage**

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       8         8         0   100.00%

================================Statement Details================================

Statement Coverage for instance /\ALU_tb#DUT_ALU  --
```

❖ Toggle coverage

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         44        44         0   100.00%

================================Toggle Details================================

Toggle Coverage for instance /\ALU_tb#DUT_ALU  --

                                          Node      1H->0L      0L->1H                          "Coverage"
                                          ----------------------------------------------------------------
                                         A[0-3]          1           1                              100.00
                                     Alu_out[4-0]        1           1                              100.00
                                         B[0-3]          1           1                              100.00
                                         C[4-0]          1           1                              100.00
                                      Opcode[0-1]        1           1                              100.00
                                            clk          1           1                              100.00
                                          reset          1           1                              100.00

Total Node Count     =         22
Toggled Node Count   =         22
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (44 of 44 bins)


Total Coverage By Instance (filtered view): 100.00%
```

3. FSM

- **Design**

  Design is correct and no bugs detected.

- **Verification plan**

Label	Design Requirement Description	Stimulus Generation	Functional Coverage	Functionality Check
FSM_1	When the reset is asserted, the output should be low	Direct at the start of the sim, then randomize with constraint that drive the reset to be off most of the simulation time	-	a checker in the testbench to make sure the output is correct
FSM_2	Try to randomize all the inputs for a while to and check the output against a golden model	Randomization under constrains on reset to be OFF most of the time	-	output checked against golden model

- **Package**

```
package FSM_PACKAGE;
    typedef enum logic [1:0] { IDLE, ZERO, ONE, STORE } state_e;
    class fsm_transaction;
        //? variables
        rand bit rst;
        rand bit x;
        state_e state;
        bit y_exp;
        bit [9:0] user_count_exp;
        //? constructor
        function new();
            rst = 0;
            x = 0;
            y_exp = 0;
            user_count_exp = 0;
            state = IDLE;
        endfunction
        //? constraints
        constraint c_dist {
            rst  dist {0:=98, 1:=2};
            x    dist {0:=67, 1:=33};
        }
    endclass
endpackage
```

- **Testbench**

```
import FSM_PACKAGE::*;
module FSM_tb ();
  //? local parameters
  localparam CLK_CYCLE = 2;
  //? class object handler creation
  fsm_transaction fsm_obj;
  //? Interface signals
  logic rst, clk, x, y;
  logic [9:0] users_count;
  //? Variables
  int error_count, correct_count;
  //? DUT instantiation
  FSM_010 DUT_FSM (
      .clk(clk),
      .rst(rst),
      .x(x),
      .y(y),
      .users_count(users_count)
  );
  //? CLK generation
  always #(CLK_CYCLE / 2) clk = ~clk;
  //? Driver block
  initial begin
    //* initialization
    clk = 0;
    fsm_obj = new();
    //* reset test
    fsm_obj.rst = 1;
    rst = fsm_obj.rst;
    check_result();
    //* randomization test
    for (int i = 0; i < 10000; i++) begin
      fsm_obj.randomize();
      x   = fsm_obj.x;
      rst = fsm_obj.rst;
      check_result();
    end
    $display("error_counts = %0d, correct_counts = %0d", error_count, correct_count);
    $stop;
  end
  //? TASKS
  task check_result();
    logic y_exp;
    logic [9:0] users_count_exp;
    golden_model(y_exp, users_count_exp);
    #CLK_CYCLE;
    if (y_exp == y && users_count_exp == users_count) correct_count++;
    else begin
      error_count++;
      $display(
          "xxxx ERROR at time %0t: the actual output result is not matching the expected result xxxx",
          $time);
      $display("y=%b ,users_count=%0d, y_exp=%b, users_count_exp=%0d", y, users_count, y_exp,
                users_count_exp);
    end
  endtask
```

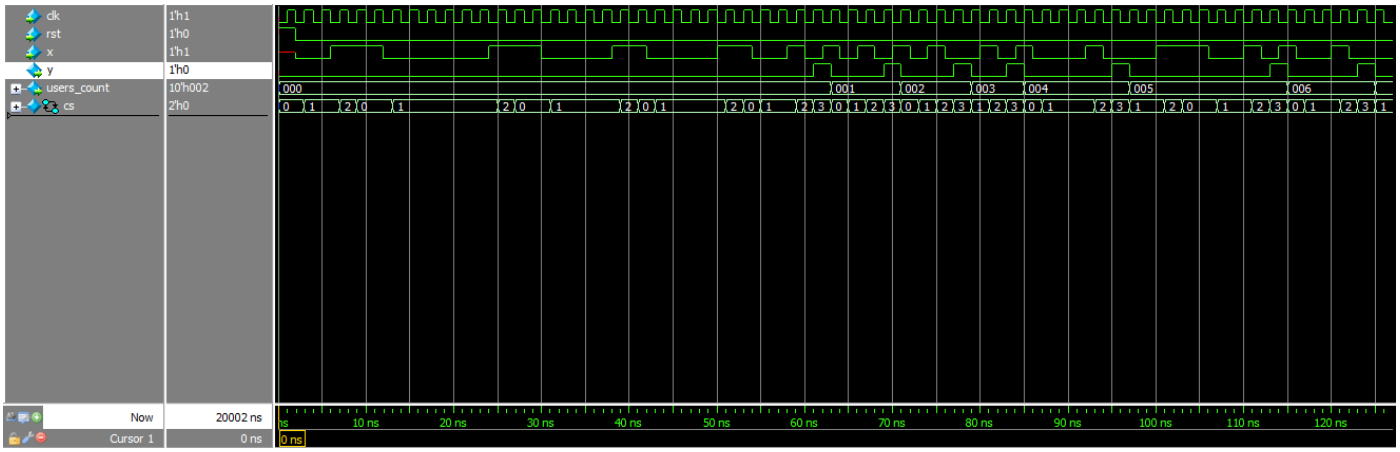
```
  task golden_model(output y_exp, output [9:0] count_exp);
    if (rst) begin
      fsm_obj.y_exp = 0;
      fsm_obj.user_count_exp = 0;
      fsm_obj.state = IDLE;
    end else begin
      case (fsm_obj.state)
        IDLE: if (x) fsm_obj.state = IDLE; else fsm_obj.state = ZERO;
        ZERO: if (x) fsm_obj.state = ONE; else fsm_obj.state = ZERO;
        ONE: if (x) fsm_obj.state = IDLE; else fsm_obj.state = STORE;
        STORE: begin
          fsm_obj.user_count_exp++;
          if (x) fsm_obj.state = IDLE; else fsm_obj.state = ZERO;
        end
        default: fsm_obj.state = IDLE;
      endcase
      if(fsm_obj.state == STORE) fsm_obj.y_exp = 1; else fsm_obj.y_exp = 0;
    end
    y_exp = fsm_obj.y_exp;
    count_exp = fsm_obj.user_count_exp;
  endtask
endmodule
```

- **Do file**

```
vlib work
vlog FSM_PACKAGE.sv FSM_010.v FSM_tb.sv +cover -covercells
vsim -voptargs=+acc work.FSM_tb -cover
coverage save FSM.ucdb -onexit -du FSM_010
#do wave.do
log -r /*
run -all
coverage exclude -src FSM_010.v -line 42 -code s
coverage exclude -src FSM_010.v -line 42 -code b
coverage exclude -du FSM_010 -ftrans cs ZERO->IDLE
coverage exclude -du FSM_010 -togglenode {users_count[6]}
coverage exclude -du FSM_010 -togglenode {users_count[7]}
coverage exclude -du FSM_010 -togglenode {users_count[8]}
coverage exclude -du FSM_010 -togglenode {users_count[9]}
quit -sim
vcover report FSM.ucdb -details -annotate -all -output cover_rpt.txt
```

- **Simulation results**

❖ **FSM_1 & FSM_2**



- **Code coverage**

  ➔ The default branch in the FSM case statement had been excluded.

  ➔ FSM from ZERO to IDLE state had been excluded as it is a wrong transition and not located in the design FSM diagram.

  ➔ Toggle nodes users_count[9:6] had been excluded, it is ok to keep them but it will take a large number of iterations to be covered. In larger designs this will cost a lot of time and computational power.

❖ **Branch coverage**

```
Coverage Report by instance with details

================================================================================
=== Instance: /\FSM_tb#DUT_FSM
=== Design Unit: work.FSM_010
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        20        20         0   100.00%

================================Branch Details================================

Branch Coverage for instance /\FSM_tb#DUT_FSM
```

❖ **FSM coverage**

```
FSM Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    FSM States                       4         4         0   100.00%
    FSM Transitions                  6         6         0   100.00%

================================FSM Details================================

FSM Coverage for instance /\FSM_tb#DUT_FSM  --

FSM_ID: cs
    Current State Object : cs
    ----------------------
    State Value MapInfo :
    ---------------------
Line          State Name              Value
----          ----------              -----
  22                IDLE                  0
  27                ZERO                  1
  32                 ONE                  2
  37               STORE                  3
    Covered States :
    ----------------
                   State          Hit_count
                   -----          ---------
                    IDLE               1709
                    ZERO               3076
                     ONE               1744
                   STORE               1144
    Covered Transitions :
    ---------------------
Line           Trans_ID          Hit_count          Transition
----           --------          ---------          ----------
  26                  0               1112          IDLE -> ZERO
  29                  1               1744          ZERO -> ONE
  36                  3               1144          ONE -> STORE
  34                  4                600          ONE -> IDLE
  41                  5                742          STORE -> ZERO
  39                  6                402          STORE -> IDLE


    Summary                       Bins      Hits    Misses  Coverage
    -------                       ----      ----    ------  --------
        FSM States                   4         4         0   100.00%
        FSM Transitions              6         6         0   100.00%
```

❖ Statement coverage

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                      16        16         0   100.00%

==============================Statement Details==============================

Statement Coverage for instance /\FSM_tb#DUT_FSM  --

    Line         Item                      Count     Source
    ----         ----                      -----     ------
  File FSM_010.v
    8                                                module FSM_010(clk, rst, x, y, users_count);
```

❖ Toggle coverage

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         28        28         0   100.00%

==============================Toggle Details==============================

Toggle Coverage for instance /\FSM_tb#DUT_FSM  --

                                          Node      1H->0L      0L->1H  "Coverage"
                                          ----------------------------------------
                                           clk           1           1      100.00
                                        cs[1-0]          1           1      100.00
                                        ns[1-0]          1           1      100.00
                                           rst           1           1      100.00
                                  users_count[5-0]       1           1      100.00
                                             x           1           1      100.00
                                             y           1           1      100.00

Total Node Count     =         14
Toggled Node Count   =         14
Untoggled Node Count =          0

Toggle Coverage      =     100.00% (28 of 28 bins)


Total Coverage By Instance (filtered view): 100.00%
```